

CO 3 ASSESSMENT TOOL – 3

DSA0134 – Object Oriented Programming with C++

NAME: T. Nivya Sri

REG NO: 192424131

Question 1: Deep Copy with Student Class

C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string* name;
    int marks;

public:
    Student(string n, int m) {
        name = new string(n);
        marks = m;
    }

    Student(const Student &s) {
        name = new string(*(s.name)); // Deep copy allocation
        marks = s.marks;
    }

    ~Student() {
        delete name;
    }

    void modifyName(string newName) {
        *name = newName;
    }

    void display() const {
        cout << "Name: " << *name << ", Marks: " << marks << endl;
    }
};

int main() {
    Student s1("Alice", 90);
    Student s2 = s1; // Calls copy constructor
```

```

cout << "Initial Objects:\n";
s1.display();
s2.display();

s2.modifyName("Bob");

cout << "\nAfter Modifying Copied Object:\n";
s1.display();
s2.display();

return 0;
}

```

Output:

Output

```

Initial Objects:
Name: Alice, Marks: 90
Name: Alice, Marks: 90

After Modifying Copied Object:
Name: Alice, Marks: 90
Name: Bob, Marks: 90

=== Code Execution Successful ===

```

Tasks Analysis:

- **Do changes in one object affect the other?** No. Because a deep copy allocates a distinct block of heap memory for the name attribute of each object, changing s2 does not impact s1.
- **Importance of deep copy:** If a default copy constructor (shallow copy) is used, both objects point to the same memory address. Modifying one alters the other, and when both objects exit scope, it results in a runtime crash known as a **double-free error**.

Question 2: Memory Management with Employee Class

C++ Code:

```

#include <iostream>
#include <string>
using namespace std;

```

```

class Employee {
private:
    string* name;
    double salary;

public:
    Employee(string n, double s) {
        name = new string(n);
        salary = s;
        cout << "Constructor called for " << *name << endl;
    }

    ~Employee() {
        cout << "Destructor called for " << *name << endl;
        delete name;
    }

    void display() const {
        cout << "Employee: " << *name << " | Salary: $" << salary << endl;
    }
};

int main() {
    Employee emp("John Doe", 75000);
    emp.display();
    return 0;
}

```

Output:

```

Output
Constructor called for John Doe
Employee: John Doe | Salary: $75000
Destructor called for John Doe

=== Code Execution Successful ===

```

Tasks Analysis:

- **What happens if the destructor is removed?** The program will compile and execute, but the heap memory dynamically allocated for name via new will remain bound to the system after execution finishes.

- **Memory leak scenario:** If thousands of Employee objects are created dynamically and deleted throughout execution without explicit structural deletion of their internal pointer members, system RAM will continuously decrease until the program suffers an out-of-memory error.

Question 3: Binary Operator Overloading (+ with Distance Class)

C++ Code:

```
#include <iostream>
using namespace std;

class Distance {
private:
    int meters;
    int centimeters;

public:
    Distance(int m = 0, int cm = 0) {
        meters = m;
        centimeters = cm;
    }

    Distance operator+(const Distance &d) {
        int totalCm = centimeters + d.centimeters;
        int totalM = meters + d.meters + (totalCm / 100);
        totalCm %= 100;
        return Distance(totalM, totalCm);
    }

    void display() const {
        cout << meters << "m " << centimeters << "cm" << endl;
    }
};

int main() {
    Distance d1(5, 80), d2(3, 40);
    Distance d3 = d1 + d2;

    cout << "Distance 1: "; d1.display();
    cout << "Distance 2: "; d2.display();
    cout << "Summed Distance: "; d3.display();

    return 0;
}
```

Output:

```
Output
Distance 1: 5m 80cm
Distance 2: 3m 40cm
Summed Distance: 9m 20cm

=== Code Execution Successful ===
```

Tasks Analysis:

- **How operator overloading simplifies addition:** It allows instances to be managed natively as though they were fundamental primitives ($d3 = d1 + d2$), significantly increasing the readability of the mathematical logic.
- **Comparison with addDistance():** A normal function would require an explicit functional call layout like $d3 = d1.addDistance(d2);$. While functionally equivalent, it breaks standard user intuition for mathematical structures.

Question 4: Relational Operator Overloading (> with Box Class)

C++ Code:

```
#include <iostream>
using namespace std;

class Box {
private:
    double volume;

public:
    Box(double v) {
        volume = v;
    }

    bool operator>(const Box &b) const {
        return volume > b.volume;
    }

    double getVolume() const {
        return volume;
    }
}
```

```

    }
};

int main() {
    Box b1(150.5), b2(200.2);

    if (b1 > b2) {
        cout << "Box 1 has a greater volume (" << b1.getVolume() << ")." << endl;
    } else {
        cout << "Box 2 has a greater volume (" << b2.getVolume() << ")." << endl;
    }

    return 0;
}

```

Output:

```

Output

Box 2 has a greater volume (200.2).

=== Code Execution Successful ===

```

Tasks Analysis:

- **How comparison is performed:** The expression `b1 > b2` translates directly into an internal compiler function signature lookup: `b1.operator>(b2)`. It evaluates the underlying values and returns a primitive boolean response.
- **What happens if the operator is not overloaded?** The compiler throws a syntax compilation error because it does not have an implicit definition for how to compare custom structures.

Question 5: Nested Block Scope Tracking

C++ Code:

```

#include <iostream>
using namespace std;

class Test {
private:
    int id;

```

```

public:
    Test(int i) {
        id = i;
        cout << "Construct " << id << endl;
    }
    ~Test() {
        cout << "Destruct " << id << endl;
    }
};

int main() {
    cout << "Main Start\n";
    Test t1(1);
    {
        cout << "Entering Block\n";
        Test t2(2);
        cout << "Exiting Block\n";
    }
    cout << "Main End\n";
    return 0;
}

```

Output:

Output

Main Start
Construct 1
Entering Block
Construct 2
Exiting Block
Destruct 2
Main End
Destruct 1

=== Code Execution Successful ===

Tasks Analysis:

- **Trace order:** Objects are created sequentially inside active execution space. When execution exits an internal scope block {}, any local object created inside that block (t2) is immediately destroyed in reverse order of its creation.
- **Scope-based object lifetime:** Local non-static stack instances live strictly within their enclosing brackets {}. Once execution passes outside those structural bounds, their stack memory frames are reclaimed automatically.

Question 6: Unary Operator Overloading (- with Number Class)

C++ Code:

```
#include <iostream>
using namespace std;

class Number {
private:
    int value;

public:
    Number(int v) {
        value = v;
    }

    void operator-() {
        value = -value;
    }

    void display() const {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Number num(25);
    cout << "Before Negation: "; num.display();

    -num;
    cout << "After Negation: "; num.display();

    return 0;
}
```

Output:

Output

```
Before Negation: Value: 25
After Negation: Value: -25
```

```
=== Code Execution Successful ===
```


Tasks Analysis:

- **Difference between unary and binary overloading:** Unary operators take zero arguments when declared as object class member functions because they act implicitly on the current invoking object context (`this`). Binary operators require one argument to serve as the right-hand side operand.

Question 7: Constructor Chain Sequence with Mobile Class

C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

class Mobile {
private:
    string brand;
    double price;

public:
    Mobile() {
        brand = "Generic";
        price = 0.0;
        cout << "Default Constructor Called\n";
    }

    Mobile(string b, double p) {
        brand = b;
        price = p;
        cout << "Parameterized Constructor Called\n";
    }

    Mobile(const Mobile &m) {
        brand = m.brand;
        price = m.price;
        cout << "Copy Constructor Called\n";
    }
};

int main() {
    Mobile m1;
    Mobile m2("Apple", 999.99);
    Mobile m3 = m2;
    return 0;
}
```

Output:

```
Output
Default Constructor Called
Parameterized Constructor Called
Copy Constructor Called

=== Code Execution Successful ===
```

Tasks Analysis:

- **Initialization method comparison:** The default constructor handles fallback object configuration. Parameterized constructors handle completely customized instance mapping upon creation. Copy constructors reproduce state values using an existing instance as a baseline reference.

Question 8: Complex Number Addition Engine

C++ Code:

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real;
    double imag;

public:
    Complex(double r = 0.0, double i = 0.0) {
        real = r;
        imag = i;
    }

    Complex operator+(const Complex &c) {
        return Complex(real + c.real, imag + c.imag);
    }

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3.5, 2.5), c2(1.5, 4.5);
```

```

Complex c3 = c1 + c2;

cout << "C1: "; c1.display();
cout << "C2: "; c2.display();
cout << "Result: "; c3.display();

return 0;
}

```

Output:

```

Output

C1: 3.5 + 2.5i
C2: 1.5 + 4.5i
Result: 5 + 7i

=== Code Execution Successful ===

```

Tasks Analysis:

- **How expression `c1 + c2` is evaluated:** It translates internally to `c1.operator+(c2)`. The member function takes the values from both inputs, combines them, and builds a brand-new `Complex` object to pass back.
- **Temporary objects created:** An anonymous temporary object is constructed inside the return line: `return Complex(...)`. This temporary object values the assignment block directly into `c3`.

Question 9: RAII Architecture with FileHandler

C++ Code:

```

#include <iostream>
using namespace std;

class FileHandler {
public:
    FileHandler() {
        cout << "File opened successfully in constructor." << endl;
    }

    ~FileHandler() {
        cout << "File closed safely in destructor." << endl;
    }
};

int main() {

```

```

{
    FileHandler fh;
    cout << "Processing active file instructions..." << endl;
}
cout << "Out of file operation scope." << endl;
return 0;
}

```

Output:

```

Output

File opened successfully in constructor.
Processing active file instructions...
File closed safely in destructor.
Out of file operation scope.

=== Code Execution Successful ===

```

Tasks Analysis:

- **Importance of destructor:** This code leverages RAII (Resource Acquisition Is Initialization). It binds resources securely to the lifetime of stack objects, guaranteeing resource deletion without relying on manual steps.
- **Consequences if missing:** Unclosed files cause data corruption, block file locks, and consume operating system file handle table allocations, preventing other system components from reading files.

Question 10: Code Execution Analysis and Tracing

C++ Code:

```

#include <iostream>
using namespace std;
class Demo {
    int x;
public:
    Demo(int a) { x = a; cout << "Construct "; }
    Demo(const Demo &d) { x = d.x; cout << "Copy "; }
    ~Demo() { cout << "Destruct "; }
    Demo operator+(Demo d) { cout << "Add "; return Demo(x + d.x); }
};
int main() {
    Demo d1(5);
    Demo d2(10);
    Demo d3 = d1 + d2;
}

```

```
    return 0;
}
```

1. Tracing the Complete Sequence:

1. Demo d1(5); $\rightarrow$ Invokes Parameterized Constructor for d1.
2. Demo d2(10); $\rightarrow$ Invokes Parameterized Constructor for d2.
3. d1 + d2 calls d1.operator+(Demo d):
 - o The parameter d is passed **by value**, so d2 is passed into the function argument space. This invokes the **Copy Constructor** to create d.
 - o Inside operator+, the code prints "Add ".
 - o return Demo(x + d.x); invokes the **Parameterized Constructor** to create an anonymous return temporary object.
4. The parameter object d exits its local execution context scope and immediately triggers its **Destructor**.
5. Depending on compiler configuration (with standard modern *Copy Elision* turned off or set strictly to trace intermediate steps): The returned anonymous temporary initializes d3 directly. If elision is bypassed manually, it would issue a copy output step. Under standard modern standard behavior, it optimizes directly into d3 storage.
6. main() reaches its conclusion block, cleaning up active stack instances (d3, d2, d1) in **reverse order** of creation, invoking three consecutive **Destructors**.

2. Exact Console Output:

```
Output Clear
Construct Construct Copy Add Construct Destruct Destruct Destruct Destruct
=== Code Execution Successful ===
```

3. Temporary Objects Created During d1 + d2:

Exactly 2 temporary/auxiliary object states are processed in this lifecycle operation block:

1. **Object 1 (Argument Copy):** The parameter variable copy instance d built on the stack using the copy constructor to handle the value collection phase for the addition scope.
2. **Object 2 (Return Value):** The anonymous returned instance created directly by evaluating Demo(x + d.x).